

Exercícios Complementares — Comunicação Interprocessos — Respostas

1. As regiões críticas do código estão no acesso à fila da requisições, encapsuladas nas funções `handoff_work()` (linha 5 no original) e `get_request()` (linha 14). Poderia ter um problema com a função `empty()` (linha 12), mas o enunciado diz que ele deve ser desconsiderado. A solução está abaixo:

```
1 semaphore mutex = 1;
2
3 void despachante() {
4     buf_t buf;
5     while (TRUE) {
6         get_next_request(&buf); // le próxima requisição, armazena em buf
7         down(&mutex);
8         handoff_work(&buf);      // insere buf na fila de requisições
9         up(&mutex);
10    }
11 }
12
13 void operaria(int i) {
14     buf_t buf;
15     while (TRUE) {
16         while (empty(req_queue)) // se a fila estiver vazia,
17             sleep(1);           // dá chance para despachante
18         down(&mutex);
19         get_request(&buf);       // pega requisição da fila
20         up(&mutex);
21         look_for_page_in_cache(&buf, &page); // procura página no cache
22         if (page_not_in_cache(&page)) // se não encontrar...
23             read_page_from_disk(&buf, &page); // lê página do disco
24         return_page(&page);      // retorna página para usuário
25     }
26 }
```

2. A refatoração do código introduziu uma nova estrutura de dados compartilhada, a fila de respostas, que é acessada pelas funções `insert_page()` e `wait_for_page()`. Como as filas de requisições e de respostas são independentes, suas regiões críticas não são relacionadas, e a melhor solução é usar um semáforo binário para cada fila. Na solução abaixo, `mutex` garante exclusão mútua no acesso à fila de requisições, e `mtx2` no acesso à fila de respostas.

```
1 semaphore mutex = 1, mtx2 = 1;
2
3 void despachante() {
4     buf_t buf;
5     while (TRUE) {
6         get_next_request(&buf); // le próxima requisição, armazena em buf
7         down(&mutex);
8         handoff_work(&buf);      // insere buf na fila de requisições
9         up(&mutex);
10    }
11 }
12
13 void operaria(int i) {
14     buf_t buf;
15     while (TRUE) {
16         while (empty(req_queue)) // se a fila estiver vazia,
17             sleep(1);           // dá chance para despachante
18         down(&mutex);
19         get_request(&buf);       // pega requisição da fila
20         up(&mutex);
21         look_for_page_in_cache(&buf, &page); // procura página no cache
22         if (page_not_in_cache(&page)) // se não encontrar...
23             read_page_from_disk(&buf, &page); // lê página do disco
24         down(&mtx2);
25         insert_page(&page);      // insere página na fila
26         up(&mtx2);
27     }
28 }
29
30 void respondente(void) {
31     page_t page;
32     while (TRUE) {
33         while (empty(page_queue)) // se a fila estiver vazia,
34             sleep(1);           // dá chance para despachante
35         down(&mtx2);
36         wait_for_page(&page);    // pega página da fila
37         up(&mtx2);
38         return_page(&page);      // retorna página para usuário
39     }
40 }
```

- 3.

<code>int n = 1; /* variável compartilhada entre X, Y e Z */</code>		
<code>semaphore S1 = 0, S2 = 0; /* semáforos compartilhados */</code>		
<pre>void X() { n = n * 16; up(&S1); }</pre>	<pre>void Y() { down(&S2); n = n / 7; }</pre>	<pre>void Z() { down(&S1); n = n + 40; up(&S2); }</pre>

4. Neste problema, os laços das linhas 13–14 e 26–27 devem ser substituídos por condições de sincronização usando semáforos. Como as filas são independentes, são usados dois semáforos contadores `sem_req` e `sem_page`, um para cada fila; cada semáforo conta o número de elementos na fila. A solução fica a seguinte:

```
1 semaphore mutex = 1, mtx2 = 1;          // semáforos binários
2 semaphore sem_req = 0, sem_page = 0;    // semáforos contadores
3
4 void despachante() {
5     buf_t buf;
6     while (TRUE) {
7         get_next_request(&buf); // le próxima requisição, armazena em buf
8         down(&mutex);
9         handoff_work(&buf);      // insere buf na fila de requisições
10        up(&mutex);
11        up(&sem_req);             // sinaliza operária
12    }
13 }
14
15 void operaria(int i) {
16     buf_t buf;
17     while (TRUE) {
18         down(&sem_req);          // bloqueia se fila reqs vazia
19         down(&mutex);
20         get_request(&buf);       // pega requisição da fila
21         up(&mutex);
22         look_for_page_in_cache(&buf, &page); // procura página no cache
23         if (page_not_in_cache(&page)) // se não encontrar...
24             read_page_from_disk(&buf, &page); // lê página do disco
25         down(&mtx2);
26         insert_page(&page);      // insere página na fila
27         up(&mtx2);
28         up(&sem_page);           // sinaliza respondente
29     }
30 }
31
32 void respondente(void) {
33     page_t page;
34     while (TRUE) {
35         down(&sem_page);        // bloqueia se fila de páginas vazia
36         down(&mtx2);
37         wait_for_page(&page);   // pega página da fila
38         up(&mtx2);
39         return_page(&page);     // retorna página para usuário
40     }
41 }
```

5. Na solução, o vetor `lugares` (linha 4) representa o mapa de alocação dos lugares. O semáforo `livres` (linha 3) conta o número de lugares livres; o `down()` na linha 9 faz com que os processos fiquem bloqueados quando todos os lugares estiverem ocupados, enquanto o `up()` na linha 21 libera um lugar caso o pagamento não seja autorizado. As regiões críticas neste código são os trechos em que o mapa de alocação é modificado (RC 1: linhas 11–14, RC 2: linha 19), e são guardadas pelo semáforo `mtx` (definido na linha 4, e usado nas linhas 10, 15, 18 e 20). Como a autorização de pagamento (linhas 16 e 17) é lenta e sujeita a falhas, ela deve estar fora da região crítica.

```
1  enum { LIVRE, OCUP };
2  semaphore mtx = 1;
3  semaphore livres = MAX;
4  int lugares[MAX] = LIVRE;
5
6  void quiosque() {
7      int lug, pg_ok;
8      while (TRUE) {
9          down(&livres);
10         down(&mtx);
11         do {
12             lug = escolhe_lugar();
13         } while (lugares[lug] == OCUP);
14         lugares[lug] = OCUP;
15         up(&mtx);
16         pg_ok = autoriza_pagamento();
17         if (!pg_ok) {
18             down(&mtx);
19             lugares[lug] = LIVRE;
20             up(&mtx);
21             up(&livres);
22         } else {
23             imprime_bilhete(lug);
24         }
25     }
26 }
```